

Practical: 1

29/07/2026

Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

Aim: To Implement and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

Algorithm:

* Algorithms :

i) Bubble sort :

Input : Unsorted Array
Output : Sorted Array

Step 1 : For $i=0$ in condition $i < n$ then do
Step 2 : For $j=0$ in condition of $j < n-i-1$ then do
Step 3 : If $arr[i] > arr[j+1]$ then swap
 temp = arr[i]
 arr[i] = arr[j+1]
 arr[j+1] = temp
Step 4 : Increment the inner loop of iteration
Step 5 : Increment the Outer loop of iteration
Step 6 : Stop.

ii) Selection sort :

Input : Unsorted Array
Output : Sorted Array

Step 1 : For $i=0$ in the Condition $i < n$ then do
Step 2 : min = i and inner for loop initialize $j=i+1$ in
Step 3 : in the Condition $j < n$ then do
 if $arr[j] > arr[min]$ then
 min = j
Step 4 : increment the inner loop $j++$
Step 5 : if min != i then
 swap (arr[i], arr[min])
Step 6 : increment the Outer loop $i++$
Step 7 : Stop.

iii) Insertion sort :

Input : Unsorted Array
Output : Sorted Array

Step 1 : For $i=1$ in the Condition $i < n$ then do
Step 2 : temp = arr[i] and $j=i-1$
Step 3 : While $j > 0$ and $arr[j] > temp$ then do
 arr[j+1] = arr[j]
 j = j-1
Step 4 : initialize arr[j+1] = temp
Step 5 : Increment the Outer loop $i++$
Step 6 : Stop.

iv) Merge sort :

Input : Unsorted Array
Output : Sorted Array

Step 1 : Check if array has more than One element
Step 2 : Divide the array into two equal halves
Step 3 : Recursively sort the Left half using merge sort
Step 4 : Same for Right half Using merge sort
Step 5 : Merge the two sorted half halves into a single sorted array
Step 6 : Stop.

v) Quick sort :

Input : Unsorted Array
Output : Sorted Array

Step 1 : Select any element as a pivot
Step 2 : partition the array by placing
 if $arr[i] < pivot$
 i++
Step 3 : place pivot in its correct sorted position
Step 4 : Recursively apply Quick sort to the
Step 5 : Left sub-array and Right sub-array
Step 7 : Stop.

Program:

1. Bubble sort, selection sort, Insertion sort

```
#include<iostream>
using namespace std;

class classname{
public:
    void Func(){
        int array1[100];
        int input,i;
        cout<<"Enter the size of array: ";
        cin>>input;
        array1[input];
        cout<<"Enter the Elements in array:\n";
        for(int i=0; i< input; i++){
            cin>>array1[i];
        }
        int n = sizeof(array1)/sizeof(array1[0]);
        display(array1,input);
        cout<<"\n\n";
        Bubble(array1,input);
        cout<<"\n\n";
        selection(array1,input);
        cout<<"\n\n";
        insertion(array1,input);
        cout<<"\n\n";
        // quickSort(array1, 0, n-1);
    }
    void display(int array1[],int n){
        cout<<"Your Array is:\n";
        for(int i = 0; i<n; i++){
            cout<<array1[i]<<" ";
        }
    }
    void Bubble(int array1[],int n){
        cout<<"Bubble sorted array:\n";
        for(int i=0;i<n;i++){
            for(int j=0; j<n-i-1; j++){
                if(array1[j] > array1[j+1]){
                    int temp = array1[j];
                    array1[j] = array1[j+1];
                    array1[j+1] = temp;
                }
            }
        }
    }
}
```

```
        }
        display(array1,n);
    }
    void selection(int array1[],int n){
        cout<<"selection sort Array:\n";
        for(int i = 0; i<n; i++){
            int min = i;
            for(int j=i+1; j<n; j++){
                if(array1[j] < array1[min]){
                    min = j;
                }
            }
            if(min != i){
                int temp = array1[i];
                array1[i] = array1[min];
                array1[min] = temp;
            }
        }
    }
    display(array1,n);
}

void insertion(int array1[],int n){
    cout<<"insertion sort Array:\n";
    for(int i=1;i<n;i++){
        int temp = array1[i];
        int j = i-1;
        while(j>=0 && array1[j]>temp){
            array1[j+1] = array1[j];
            j=j-1;
        }
        array1[j+1] = temp;
    }
    display(array1,n);
}

int main(){
    classname obj;
    obj.Func();
    return 0;
}
```

2. Quick sort and merge sort:

```
#include<iostream>
```

```
using namespace std;
```

```
//----- Swap Function -----
```

```
void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

//----- Quick Sort -----
int partition(int arr[], int start, int end) {
    int pivot = arr[end];
    int i = start - 1;

    for (int j = start; j < end; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }

    swap(&arr[i + 1], &arr[end]);
    return i + 1;
}

void quickSort(int arr[], int start, int end) {
    if (start < end) {
        int pi = partition(arr, start, end);

        quickSort(arr, start, pi - 1);
        quickSort(arr, pi + 1, end);
    }
}

//----- Merge Sort -----
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int L[n1], R[n2];

    // Copy data to temporary arrays
    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];

    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
```

```
// Merge the temporary arrays
while (i < n1 && j < n2) {
    if (L[i] <= R[j]) {
        arr[k] = L[i];
        i++;
    }
    else {
        arr[k] = R[j];
        j++;
    }
    k++;
}

// Copy remaining elements
while (i < n1) {
    arr[k] = L[i];
    i++;
    k++;
}

while (j < n2) {
    arr[k] = R[j];
    j++;
    k++;
}

}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

void display(int array1[], int n) {
    for (int i = 0; i < n; i++) {
        cout << array1[i] << " ";
    }
    cout << endl;
}

int main() {
    int arr[] = {9, 9, 7, 78, 5};
    int n = sizeof(arr) / sizeof(arr[0]);
```

```
cout << "Original Array:\n";
display(arr, n);
//Quick Sort
quickSort(arr, 0, n - 1);
    cout<<endl;
cout<<"Quicksort sorted array: ";
display(arr, n);
// Merge Sort
mergeSort(arr, 0, n - 1);
cout<<endl;
cout<<"mergesort sorted array: ";
display(arr, n);
return 0;
}
```

Output:

1.

```
Enter the size of array: 5
Enter the Elements in array:
1 9 87 4 5
Your Array is:
1 9 87 4 5

Bubble sorted array:
Your Array is:
1 4 5 9 87

selection sort Array:
Your Array is:
1 4 5 9 87

insertion sort Array:
Your Array is:
1 4 5 9 87

-----
Process exited after 11.74 seconds with return value 0
Press any key to continue . . .
```

2.

```
Original Array:
9 9 7 78 5

Quicksort sorted array: 5 7 9 9 78

mergesort sorted array: 5 7 9 9 78

-----
Process exited after 0.016 seconds with return value 0
Press any key to continue . . . |
```


Time Complexity:

Time Complexity:

1) Bubble sort:

```

for(i=0; i<n; i++) {
    for(j=0; j<n-i-1; j++) {
        if(arr[j] > arr[j+1]) {
            t = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = t;
        }
    }
}

```

Total T(n) = $(n+2) + \frac{n(n-1)}{2} + \frac{n(n-1)}{2}$

$= \frac{2n^2 - n + 2n + 2}{2}$

$= \frac{2n^2 + n + 2}{2}$

$= n^2 + \frac{n}{2} + 1$

$\approx n^2$

Time Complexity = $O(n^2)$

2) Selection sort:

```

for(i=0; i<n; i++) {
    min = i;
    for(j=i+1; j<n; j++) {
        if(arr[j] < arr[min]) {
            min = j;
        }
    }
    if(min != i) {
        t = arr[i];
        arr[i] = arr[min];
        arr[min] = t;
    }
}

```

Total T(n) = $(n+2) + \frac{n(n-1)}{2} + \frac{n(n-1)}{2}$

$= \frac{2n^2 - n + 2n + 2}{2}$

$= \frac{2n^2 + n + 2}{2}$

$= n^2 + \frac{n}{2} + 1$

$\approx n^2$

Time Complexity = $O(n^2)$

3) Insertion sort:

```

for(i=1; i<n; i++) {
    t = arr[i];
    j = i-1;
    while(j > 0 && arr[j] > t) {
        arr[j+1] = arr[j];
        j = j-1;
    }
    arr[j+1] = t;
}

```

Total T(n) = $(n+2) + \frac{n(n-1)}{2} + \frac{n(n-1)}{2}$

$= \frac{2n^2 - n + 2n + 2}{2}$

$= \frac{2n^2 + n + 2}{2}$

$= n^2 + \frac{n}{2} + 1$

$\approx n^2$

Time Complexity = $O(n^2)$

4) Merge sort:

```

Merge sort(arr, left, right) {
    if (left < right) {
        m = (left + right) / 2;
        merge sort(arr, left, m);
        merge sort(arr, m+1, right);
        merge(arr, left, m, right);
    }
}

```

Total T(n) = $O(n) \times O(\log n)$

$= O(n \log n)$

5) Quick sort:

```

Quick sort(arr, s, e) {
    if (s < e) {
        p = partition(arr, s, e);
        Quick sort(arr, s, p-1);
        Quick sort(arr, p+1, e);
    }
}

```

Total T(n) = $O(n) \times O(\log n)$

$T(n) = T(n-1) + O(n)$

$T(n-1) = T(n-2) + O(n-1)$

$T(n) = T(n-1) + 2 + 3 + 4 + \dots + n$

$T(n) = O(n^2)$

Time Complexity:

1. Bubble sort:

- Best case: $O(n)$
- Average case: $O(n^2)$
- Worst case: $O(n^2)$

2) Selection sort:

- Best case: $O(n^2)$
- Average case: $O(n^2)$
- Worst case: $O(n^2)$

3) Insertion sort:

- Best case: $O(n)$
- Worst case: $O(n^2)$
- Average case: $O(n^2)$

4) Merge sort:

- Best case: $O(n \log n)$
- Worst case: $O(n \log n)$
- Average case: $O(n \log n)$

5) Quick sort:

- Best case: $O(n \log n)$
- Average case: $O(n \log n)$
- Worst case: $O(n^2)$

Conclusion:

Hence, By Implementing and analysing of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

The time complexity plays an important role in the logical part of the program. And the Programs are successfully executed.